

第三章 处理机调度与死锁

by Jiacheng Xu

- 第三章 处理机调度与死锁
 - 3.1 处理机调度的基本概念
 - 3.1.1 操作系统分配处理机的基本流程
 - 3.1.2 高级、中级、低级三种调度
 - (1) 高级调度
 - 1. 高级调度的概念
 - 2. 高级调度的特点
 - (2) 低级调度
 - 1. 低级调度的概念
 - 2. 低级调度的特点
 - 3. 调度方式：非抢占方式
 - 4. 调度方式：抢占方式
 - (3) 中级调度
 - 1. 中级调度的概念
 - 2. 中级调度的特点
 - 3.1.3 调度队列模型
 - (1) 仅有进程调度的调度队列模型
 - (2) 具有高级调度和低级调度的调度队列模型
 - (3) 同时具有高、中、低三级调度的调度队列模型
 - 3.1.4 选择调度方式和衡量调度算法的性能
 - (1) 面向用户的准则
 - (2) 面向系统的准则
 - 3.2 调度算法
 - 3.2.0 调度算法的概念
 - 3.2.1 先来先服务调度算法 (FCFS)
 - (1) 设计思想
 - (2) 特点
 - 3.2.2 短作业/短进程优先调度算法 (SJF/SPF)
 - (1) 设计思想
 - (2) 特点
 - FCFS、SJF/SJP算法实例
 - (0) 总体思路分析

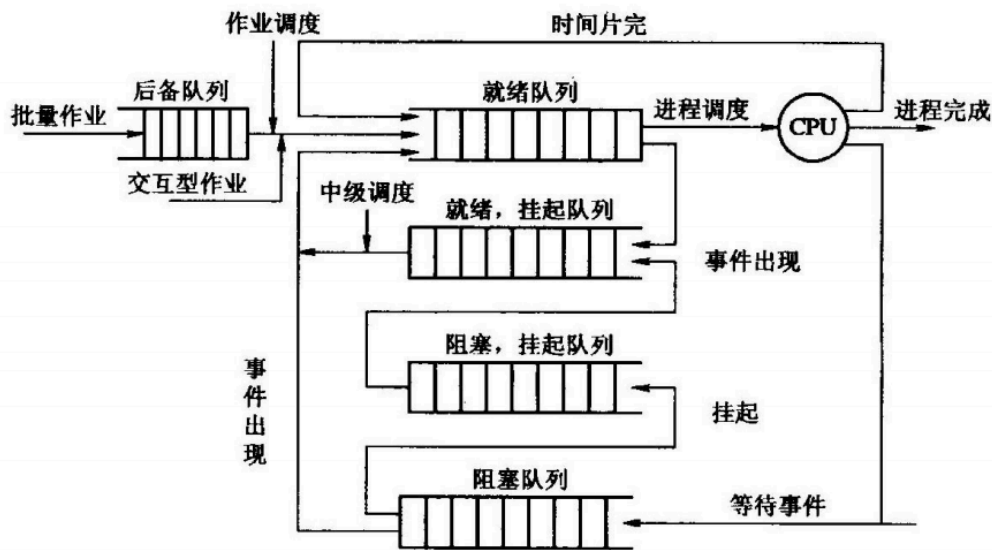
- (1) FCFS分析
 - (2) SJP分析
- 3.2.3 高优先权优先调度算法 (FPF)
 - (1) 设计思想
 - (2) 优先权
 - 1. 优先权的概念
 - 2. 进程的优先权
 - 3. 优先权的确定
- 3.2.4 高响应比优先调度算法
 - (1) 思想概述
 - (2) 特点
- 3.2.5 时间片轮转调度算法 (RR)
 - (1) 设计思想
 - (2) 时间片
 - 1. 时间片选择
 - 2. 时间片大小选择
- 3.2.6 多级反馈队列调度算法 (MFQ)
 - (1) 设计思想
 - (2) 算法步骤
 - (3) 特点
- 3.3 实时调度
 - 3.3.0 实时系统中的调度问题
 - 3.3.1 实现实时调度的基本条件
 - 3.3.2 实时调度算法的分类
 - (0) 总体分类
 - (1) 非抢占式调度算法
 - (2) 抢占式调度算法
 - 3.3.3 两个典型的实时调度算法
 - (1) 最早截止时间优先调度算法(EDF)
 - (2) 最低松弛度优先调度算法(LLF)
 - 3.3.4 优先级倒置问题
 - (1) 优先级倒置
 - (2) 优先级倒置实例
 - (3) 优先级倒置的解决方法
- 3.5 死锁的基本概念
 - 3.5.1 死锁及死锁状态
 - 3.5.2 产生死锁的原因

- 3.5.3 产生死锁的必要条件——死锁的特征
- 3.5.4 处理死锁的基本方法
 - (1) 预防死锁
 - (2) 避免死锁
 - (3) 检测和解除死锁
- 3.6 预防死锁
 - 3.6.1 预防死锁的基本概念
 - 3.6.2 破坏“互斥”条件（不可能）
 - 3.6.3 破坏“请求和保持”条件
 - 3.6.4 破坏“不剥夺”条件
 - 3.6.5 破坏“环路等待”条件
- 3.7 避免死锁
 - 3.7.0 避免死锁的基本思想
 - 3.7.1 安全状态与不安全状态
 - (1) 定义
 - (2) 示例
 - (3) 两种状态的关系
 - 3.7.2 银行家算法（算法重点）
 - (1) 问题
 - (2) 数据结构设置
 - (3) 银行家算法（进程 P_i 请求 $Request\{i\}$ ）
 - (4) 安全性算法
 - (5) 例题
 - (6) 总结
 - (7) 思考题
- 3.8 死锁的检测和解除
 - 3.8.0 基本思想
 - 3.8.1 死锁的检测
 - (1) 资源分配图
 - (2) 死锁检测
 - 3.8.2 死锁的解除

3.1 处理机调度的基本概念

3.1.1 操作系统分配处理机的基本流程

等会儿再看，现在看不懂



具有三级调度时的调度队列模型

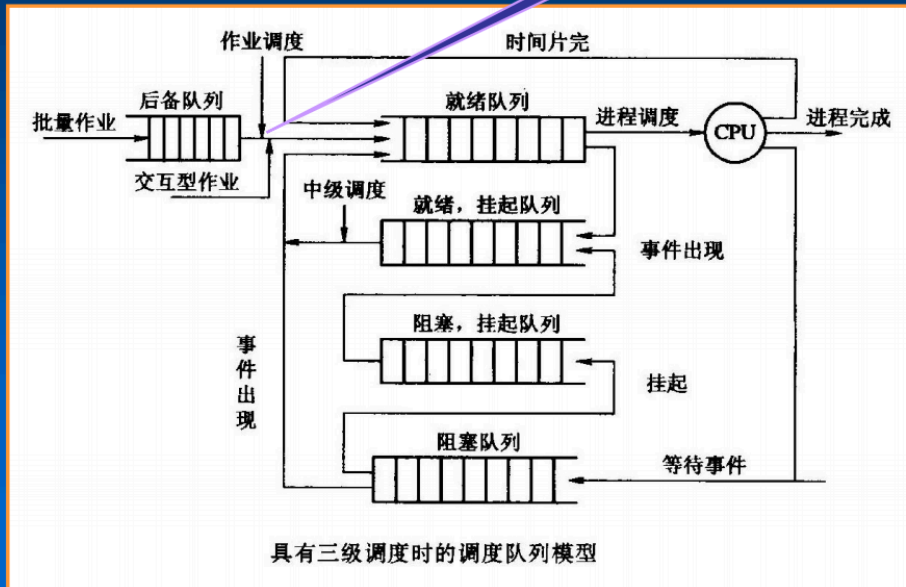
3.1.2 高级、中级、低级三种调度

(1) 高级调度

3.1 处理机调度的基本概念

3.1.2 高级、中级、低级三级调度

(1) 高级调度



1. 高级调度的概念

又名长程调度、作业调度、接纳调度

作用（注意结合图查看）：

- 判断——用于决定把外存上处于后备队列中的哪些作业调入内存

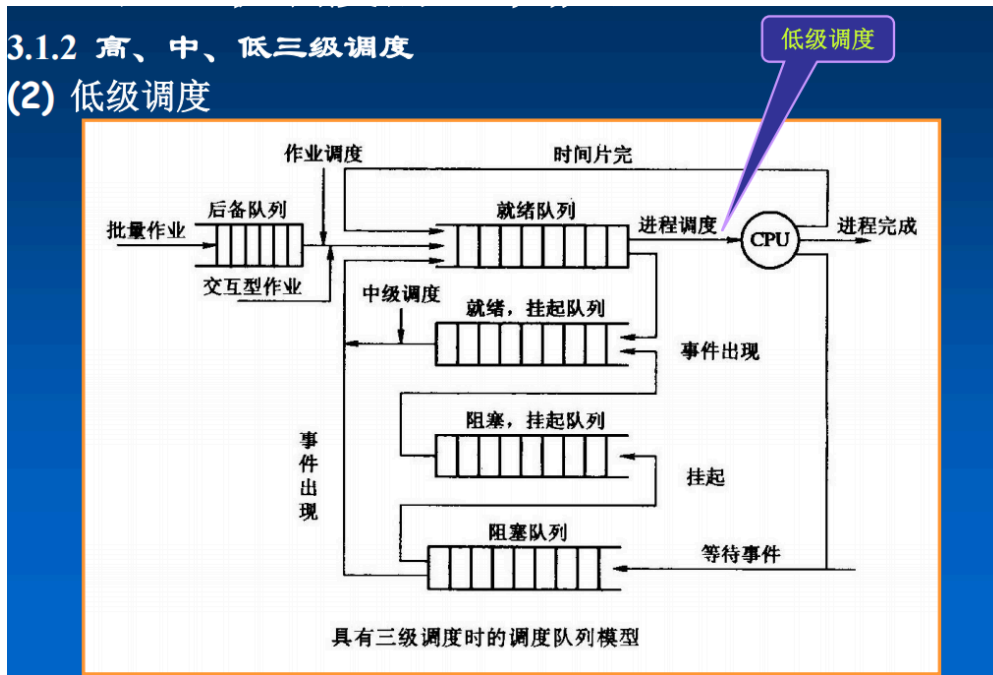
- 准备——并为它们创建进程、分配必要的资源，再将新创建的进程排在就绪队列上，准备执行。

对象：作业

2. 高级调度的特点

1. 调度频率低：几分钟或几十分钟。
2. 调度算法可以很复杂

(2) 低级调度



1. 低级调度的概念

又名进程调度、短程调度

作用 (注意结合图查看)：

- 判断——用来决定就绪队列中的哪个进程应获得处理机
- 分配——然后再由分派程序执行把处理机分配给该进程的具体操作

对象：就绪进程

2. 低级调度的特点

1. 调度频率高：几毫秒或几十毫秒。
2. 调度算法通常简单，保证算法执行时间短

3. 调度方式：非抢占方式

定义/过程：

- **无法抢占**——某一进程正在处理机上执行时，即使有某个**更为重要或紧迫**的进程进入就绪队列，仍然让正在执行的进程**继续执行**
- **结束释放处理机**——直到**该进程完成或发生某种事件而进入阻塞状态**时，才把**处理机分配**给更为重要或紧迫的进程。

别名：非剥夺方式、不可剥夺方式

优点：简单，系统开销小

缺点：实时性差，不安全——霸占CPU导致进程“饥饿”

4. 调度方式：抢占方式

定义/过程：

- **可直接抢占**——指当一个进程正在处理机上执行时，若有某个更为重要或紧迫的进程需要使用处理机，则**立即暂停正在执行的进程**，将处理机分配给这个**更为重要或紧迫的进程**。

别名：剥夺方式、可剥夺方式

优点：安全，不会霸占CPU

实现方式：

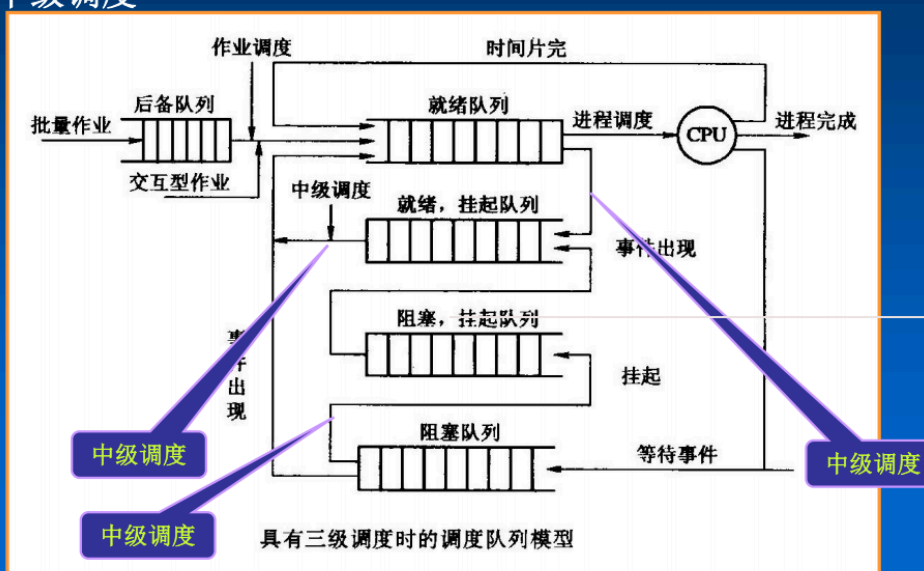
- 优先权原则
- 段作业（进程）优先
- 时间片原则

(3) 中级调度

3.1 处理机调度的基本概念

3.1.2 高、中、低三级调度

(3) 中级调度



1. 中级调度的概念

- **目的**：提高内存利用率和系统吞吐量

- **进程内存→外存：**
 - 使暂时不能运行的进程从内存调至外存上等待
 - 此时的进程状态成为就绪驻外存状态或挂起状态
- **中级调度控制重新调入内存：**
 - 进程若重新具备运行条件且内存略空闲，则中级调度决定把外存的哪些进程重新调入内存
 - 并修改其状态为就绪状态并挂在就绪队列上
- **多种状态：** 内存就绪（表示进程在内存中就绪）和 外存就绪（进程在外存中就绪），内存阻塞和外存阻塞
- **对象：** 就绪进程、阻塞进程

2. 中级调度的特点

1. 调度频率：介于高级调度和低级调度之间。
2. 实际就是内存管理的“对换”功能

3.1.3 调度队列模型

(1) 仅有进程调度的调度队列模型

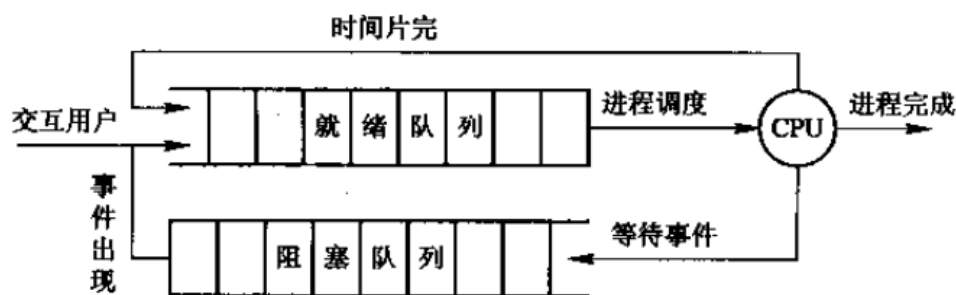


图 3 - 1 仅具有进程调度的调度队列模型

(2) 具有高级调度和低级调度的调度队列模型

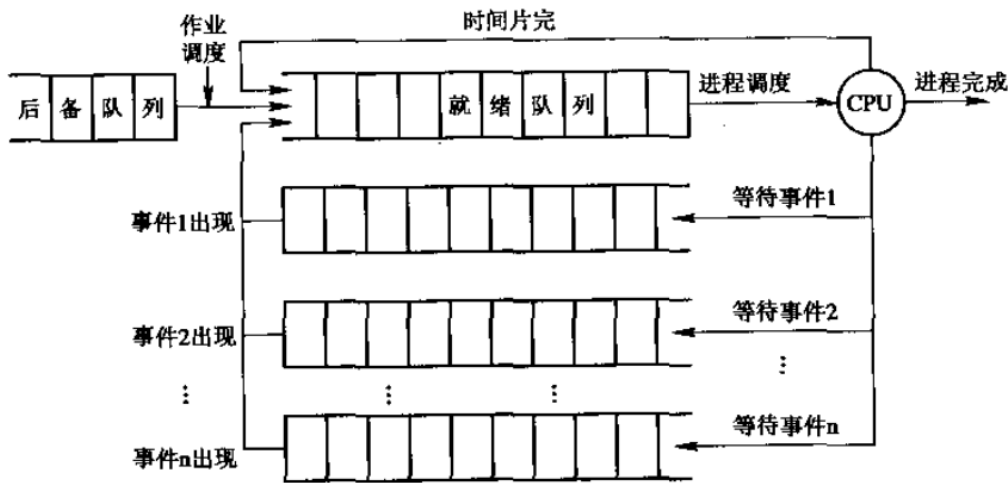
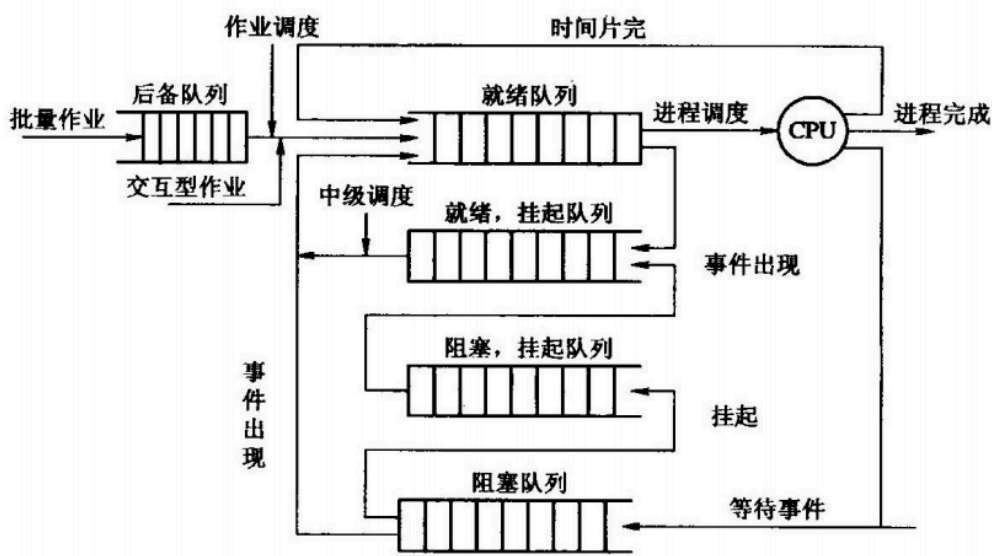


图 3-2 具有高、低两级调度的调度队列模型

(3) 同时具有高、中、低三级调度的调度队列模型



具有三级调度时的调度队列模型

3.1.4 选择调度方式和衡量调度算法的性能

(1) 面向用户的准则

1. 周转时间短

- **周转时间的定义**：从作业提交给系统开始，到作业完成为止的时间间隔（一定要理解准确，不只是处理机运行该进程的时间！！！而是提交后直到完成的全部时间！！！）
 - 作业在后备队列的等待时间；
 - 进程在就绪队列的等待时间；

- 进程在CPU上的执行时间;
- 进程等待 (如I/O操作) 时间;
- **平均周转时间**: 所有周转时间的平均值
 - $T = \frac{1}{n} \sum_{i=1}^n T_i$
- **平均带权周转时间**: 所有作业周转时间/要求服务时间的平均值
 - $W = \frac{1}{n} \sum_{i=1}^n \frac{T_i}{T_{si}}$
 - T_i 表示第*i*个作业的周转时间
 - T_{si} 表示第*i*个作业的要求服务时间

2. 响应时间快

- **响应时间**: 用户从提交键盘命令开始, 到系统首次给出响应为止的时间
- ## 3. 截止时间保证
- 截止时间: 作业/进程开始或结束的最晚时间

(2) 面向系统的准则

1. 系统吞吐量高
 - **吞吐量**: 系统单位时间内完成的作业/进程数量
2. 处理机利用率好
3. 资源利用均衡

3.2 调度算法

3.2.0 调度算法的概念

定义:

- 实质是一种资源分配方法
- 因而调度算法是指: 根据系统的资源分配策略所规定的资源分配算法

3.2.1 先来先服务调度算法 (FCFS)

(1) 设计思想

- 既可用于作业调度, 也可用于进程调度
- 用于**作业调度**:
 - 每次调度都是从后备作业队列中, 选择一个或多个最先进入该队列的作业
 - 将它们调入内存, 为它们分配资源、创建进程, 然后放入就绪队列。
- 用于**进程调度**:
 - 则每次调度是从就绪队列中, 选择一个最先进入队列的进程
 - 为之分配处理机, 使之投入运行。

(2) 特点

- 本质：只考虑“到达时间”
- 实现简单
- 貌似公平，但实际上，对短作业不公平（短作业使用资源时间短）

3.2.2 短作业/短进程优先调度算法（SJF/SPF）

(1) 设计思想

- 既可用于作业调度(SJF)，也可用于进程调度(SPF)
- 用于作业调度：
 - 每次调度都是从后备作业队列中，选择一个要求服务时间（执行时间）最短的作业，
 - 将它们调入内存，为它们分配资源、创建进程，然后放入就绪队列。
- 用于进程调度：
 - 则每次调度是从就绪队列中，选择一个执行时间最短的进程
 - 为之分配处理机，使之投入运行

(2) 特点

- 本质：仅考虑“执行时间”
- 实现困难，估算执行时间很难
- 有利于短作业，对长作业不公平

FCFS、SJF/SJP算法实例

✎ 例题

5进程A,B,C,D,E分别于0,1,2,3,4时刻到达系统，要求服务时间分别为4,3,5,2,4，请计算：

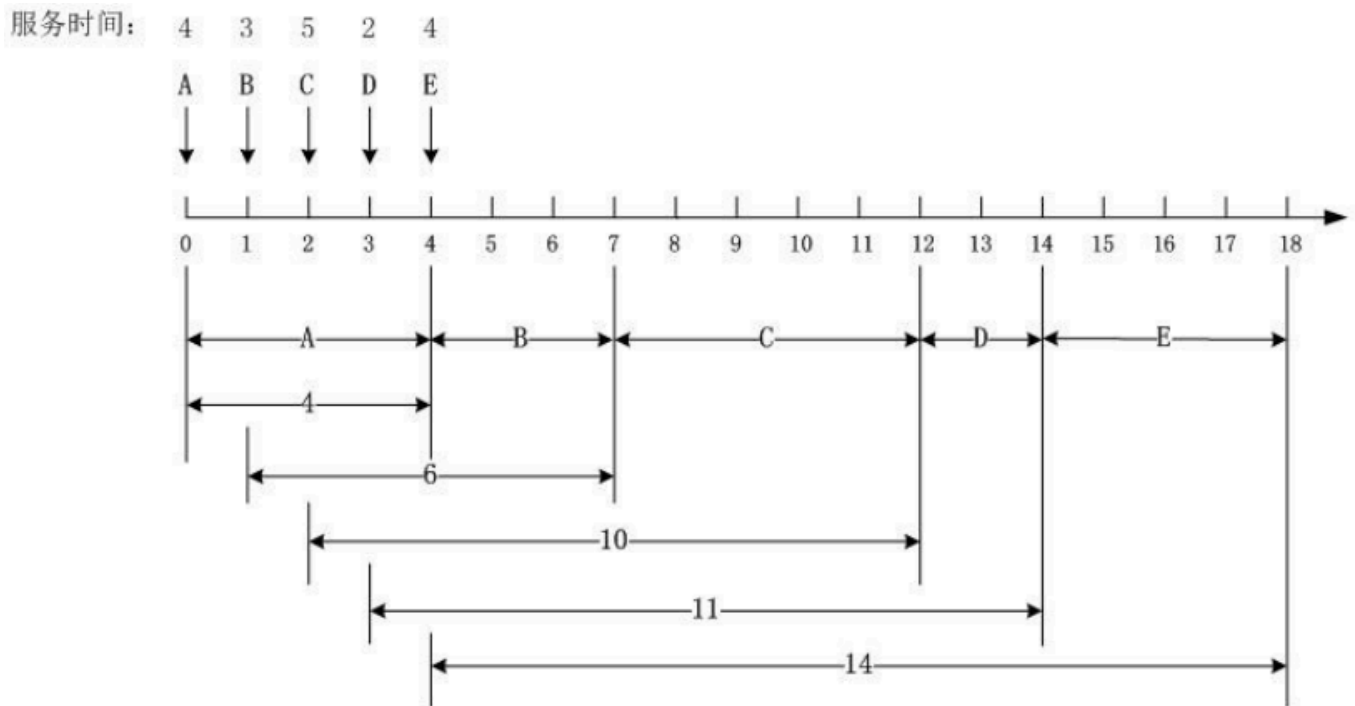
- (1)采用FCFS时进程执行顺序、平均周转时间和平均带权周转时间；
- (2)采用SPF时进程执行顺序、平均周转时间和平均带权周转时间；

(0) 总体思路分析

分析调度算法，一般画图比较直观，可以参考下面的绘图方式，进行直观地理解和分析
我推荐考试就这样画，不要用别的方法

计算时，注意周转时间的定义和平均带权周转时间计算

(1) FCFS分析



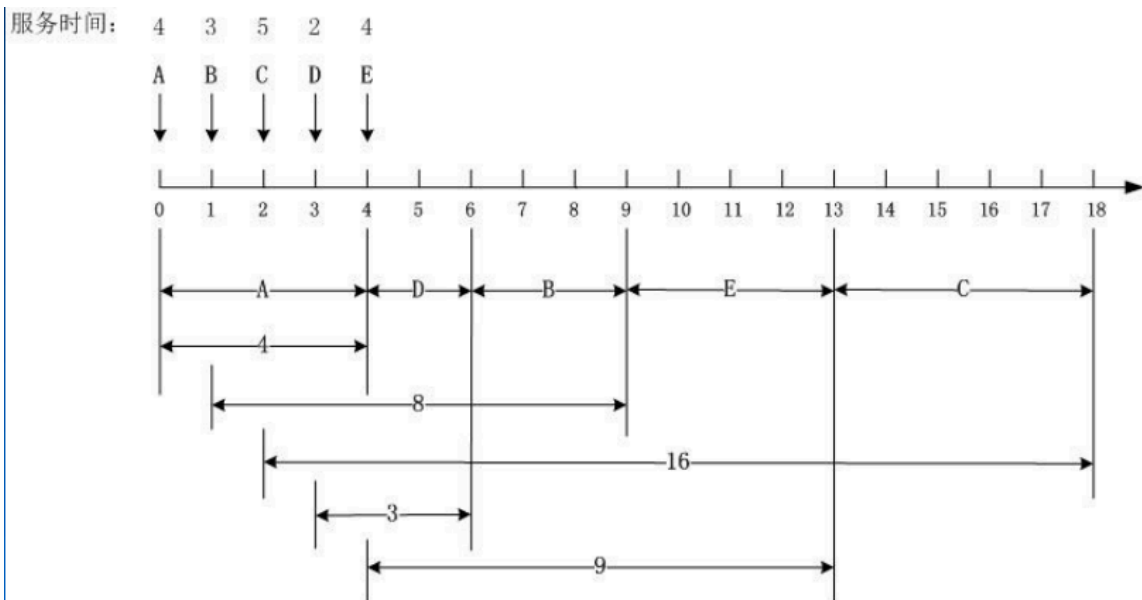
计算结果如下:

进程执行顺序: A→B→C→D→E

平均周转时间: $\frac{1}{n} \sum_{i=1}^n T_i = \frac{1}{5} \times (4 + 6 + 10 + 11 + 14) = 9$

平均带权周转时间: $\frac{1}{n} \sum_{i=1}^n \frac{T_i}{T_{si}} = \frac{1}{5} \times (\frac{4}{4} + \frac{6}{3} + \frac{10}{5} + \frac{11}{2} + \frac{14}{4}) = 2.8$

(2) SJP分析



计算结果如下:

进程执行顺序：A→D→B→E→C

平均周转时间： $\frac{1}{n} \sum_{i=1}^n T_i = \frac{1}{5} \times (4 + 8 + 16 + 3 + 9) = 8$

平均带权周转时间： $\frac{1}{n} \sum_{i=1}^n \frac{T_i}{T_{si}} = \frac{1}{5} \times (\frac{4}{4} + \frac{8}{3} + \frac{16}{5} + \frac{3}{2} + \frac{9}{4}) = 2.12$

3.2.3 高优先权优先调度算法 (FPF)

(1) 设计思想

- 既可用于**作业调度**，也可用于**进程调度**。
- 用于**作业调度**：
 - 系统将从**后备队列**中选择若干个**优先权最高的作业**，装入内存。
- 用于**进程调度**：
 - 该算法是把处理机分配给**就绪队列**中**优先权最高的进程**。
 - 分为**非抢占式优先权**算法和**抢占式优先权**调度算法

(2) 优先权

1. 优先权的概念

定义：

- 反映**作业/进程执行时的迫切程度**
- 是对调度所考虑的实际因素的算法抽象

表示：

- 通常用**1个整型数**来表示

2. 进程的优先权

优先权类型	方式
抢占式优先权	高优先权进程到达时， 立刻停止低优先权进程的执行 ，让高优先权进程执行
非抢占式优先权	高优先权作业进程到达时， 须等待低优先权进程执行完毕 或主动释放CPU

3. 优先权的确定

优先权类型	定义
静态优先权	进程创建时确定 （根据进程类型、资源需求和用户要求等），直到进程执行结束， 保持不变

优先权类型	定义
-------	----

动态优先权	进程创建时确定（根据进程类型、资源需求和用户要求等）初始优先权，在进程执行过程中，可以发生变化
-------	---

3.2.4 高响应比优先调度算法

(1) 思想概述

- 用于作业调度
- 系统将从后备队列中选择若干个响应比(优先权)最高的作业，装入内存，投入运行。

$$\text{优先权} = \frac{\text{响应时间}}{\text{要求服务时间}}$$
$$\text{优先权} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = 1 + \frac{\text{等待时间}}{\text{要求服务时间}}$$

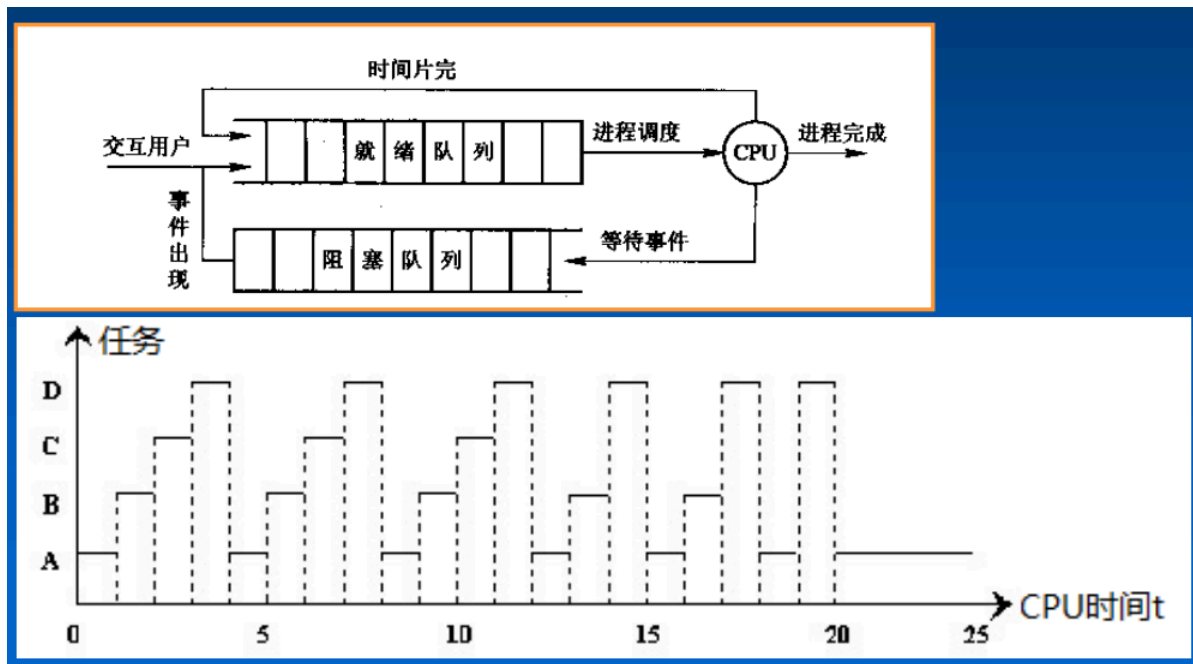
响应比 R_p

- 核心：对等待作业以一定速率 a 提高其优先权

(2) 特点

1. **短作业利好**——如果作业的等待时间相同，则要求服务的时间愈短，其优先权愈高，因而该算法有利于短作业。
2. **等待时间影响优先级**——当要求服务的时间相同时，作业的优先权决定于其等待时间，
3. **先来先服务**——等待时间愈长，其优先权愈高，因而它实现的是先来先服务。
4. **长作业等待后高优先级**——对于长作业，作业的优先级可以随等待时间的增加而提高，当其等待时间足够长时，其优先级便可升到很高，从而获得处理机。
5. **折中的优势**——既照顾了短作业，又考虑了作业到达的先后次序，不会使长作业长期得不到服务。因此，该算法实现了一种较好的折衷。
6. **缺点**——须做响应比计算，会增加系统开销

3.2.5 时间片轮转调度算法 (RR)



(1) 设计思想

- 用于作业调度或进程调度
- **进程先来先服务，时间片轮转**——在早期的时间片轮转法中，系统将所有就绪进程按先来先服务的原则，排成一个队列，每次调度时，把CPU分配给队首进程，并令其执行一个时间片。
- **用完时间片到后面站着去**——当执行的时间片用完时，由一个计时器发出时钟中断请求，调度程序便据此信号来停止该进程的执行，并将它送往就绪队列的末尾；
- **循环**——然后，再把处理机分配给就绪队列中新的队首进程，同时也让它执行一个时间片。
- **保证响应时间**：就绪队列中的所有进程，在给定时间内，均能获得一个时间片的处理机执行时间。换言之，系统能在给定的时间内，响应所有用户的请求。
- **时间片的大小**：几毫秒 到几百毫秒。

(2) 时间片

1. 时间片选择

- 固定时间片
- 可变时间片

2. 时间片大小选择

- **不可太大**：影响最大响应时间：
 - $T = nq$ ；其中， n 为进程数量， q 为时间片大小。
- **不可太小**：调度开销，增加周转时间；

3.2.6 多级反馈队列调度算法 (MFQ)

(1) 设计思想

1. **不同优先级的多个就绪队列 (优先级递减)**：设置多个就绪队列，为各队列赋予不同的优先级。第1个队列的优先级最高，其余队列优先权逐个降低。
2. **优先级高，时间片小**：赋予各个队列中进程执行时间片的大小各不相同：优先权愈高的队列中，为每个进程所规定的执行时间片就愈小。
3. **前 $i-1$ 队列空闲，才执行第 i 队列**：仅当第1队列空闲时，调度程序才调度第2队列中的进程运行；仅当第1~ $(i-1)$ 队列均空时，才会调度第 i 队列中的进程运行。
4. **高级别队列抢占**：如果处理机正在第 i 队列中为某进程服务时，又有新进程进入优先权较高的队列(第1~ $(i-1)$ 中的任何一个队列)，则此时新进程将抢占正在运行进程的处理机(正在运行进程被放回原所在队列末尾)。
5. (个人理解) FCFS、高优先级抢占式、时间片轮转算法的综合体

(2) 算法步骤

1. **新创建进入1队列FCFS**：对新创建进程，首先将它放入第1队列末尾，按FCFS原则排队等待调度。
2. **执行完成则结束，未完成进入2队列FCFS**：当轮到该进程执行时，如它能在该时间片内完成，则结束；如果未完成，调度程序便将该进程转入第2队列的末尾，再同样按FCFS原则等待调度执行；
3. **未完成则一直降级，直至第 n 队列则时间片轮转**：如果它在第2队列中运行1个时间片后仍未完成，再依次将它放入第3队列...。如此下去，当1个长作业(进程)从第1队列依次降到第 n 队列后，在第 n 队列中便采取按时间片轮转的方式运行

(3) 特点

1. 避免了事先计算各种进程所需的执行时间。
2. 具有较好的性能，能较好地满足各种类型用户需要：
 - **终端型作业用户**：终端型作业大多属于交互型作业，作业通常较小，系统只要能使这些作业(进程)在第一队列所规定的时间片内完成，便可使终端型作业用户都感到满意。
 - **短批处理作业用户**：对于很短的批处理型作业，开始时像终端型作业一样，如果仅在第一队列中执行一个时间片即可完成，便可获得与终端型作业一样的响应时间。对于稍长的作业，通常也只需在第二队列和第三队列各执行一个时间片即可完成，其周转时间仍然较短。
 - **长批处理作业用户**：对于长作业，它将依次在第1, 2, ..., n 个队列中运行，然后再按轮转方式运行，用户不必担心其作业长期得不到处理。

3.3 实时调度

3.3.0 实时系统中的调度问题

- 实时系统中都存在着若干个实时进程或任务，往往带有某种程度的紧迫性，因而对实时系统中的调度提出了某些特殊要求
- 前面所介绍的多种调度算法，并不能很好地满足实时系统对调度的要求。
- 为此，引入一种新的调度，即实时调度

3.3.1 实现实时调度的基本条件

可调度条件：对于 m 个实时任务，处理时间为 C_i ，周期时间为 P_i ，如果满足下列条件

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

则系统是可调度的

- **公式的含义：**所有实时任务，处理时间和周期时间之比的和小于等于1，则可以实现调度
- 示例1：可调度
 - A：周期：1s，处理时间：70ms
 - B：周期：1s，处理时间：90ms
 - $\sum_{i=1}^m \frac{C_i}{P_i} = \frac{0.07}{1} + \frac{0.09}{1} = \frac{0.16}{1} \leq 1$ 满足条件
- 示例2：不可调度
 - A：周期：1s，处理时间：700ms
 - B：周期：1s，处理时间：900ms
 - $\sum_{i=1}^m \frac{C_i}{P_i} = \frac{0.7}{1} + \frac{0.9}{1} = \frac{1.6}{1} > 1$ 不满足条件

3.3.2 实时调度算法的分类

(0) 总体分类

1. 根据实时任务性质
 - 硬实时调度算法：严格实时
 - 软实时调度算法：非严格实时
2. 根据调度方式
 - 非抢占调度算法
 - 抢占调度算法
3. 根据调度时间
 - 静态调度：在进程执行前，调度程序便已经决定了各进程间的执行顺序。

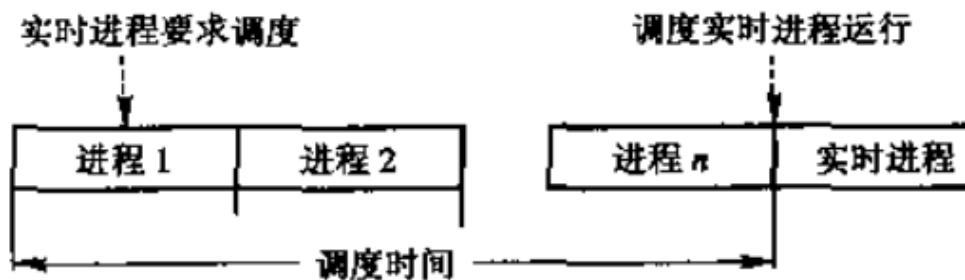
- 动态调度算法：在进程的执行过程中，由调度程序届时根据情况临时决定将哪一进程投入运行；

(1) 非抢占式调度算法

其实发现和之前的调度算法很类似

1. 非抢占式轮转调度算法

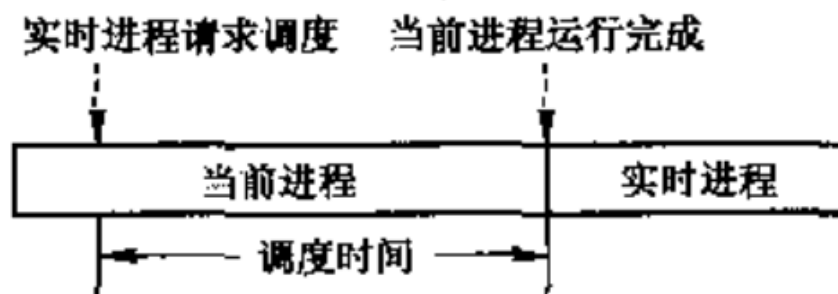
- 将所有实时任务排列成队列，轮流投入运行
- 当任务完成，就重新排到队列末尾；
- 调度时选择队首进程；
- 实时响应时间：数秒~数十秒



(a) 非抢占轮转调度

2. 非抢占式优先调度算法(更快)

- 为所有实时任务设定优先权；
- 高优先权进程到达队列，排到队首，以待当前任务执行完毕；
- 调度时选择队首进程；
- 实时响应时间：数百毫秒~数秒

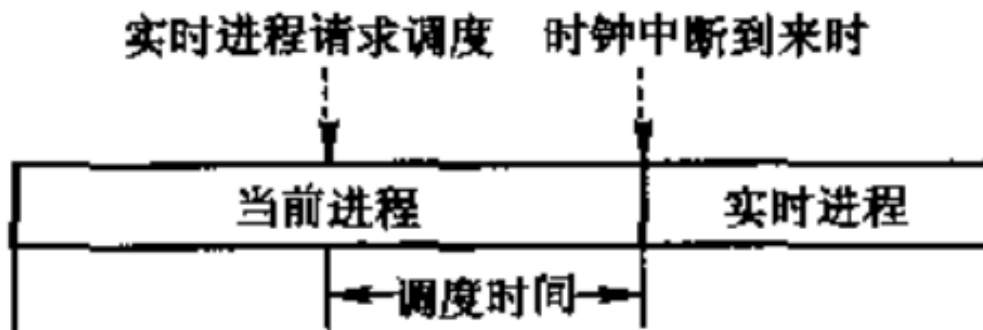


(b) 非抢占优先权调度

(2) 抢占式调度算法

1. 基于时钟中断的抢占式优先权调度算法 (更更快)

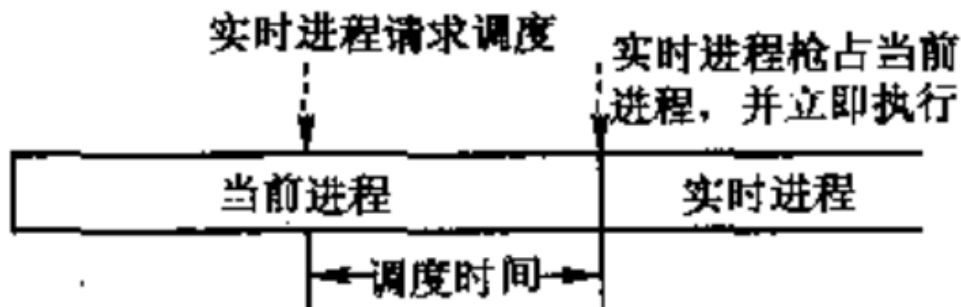
- 为所有实时任务设定优先权；
- 高优先权进程到达队列，排到队首，以待时钟中断到达时，调度到CPU上执行；
- 调度时选择队首进程；
- 实时响应时间：数毫秒-数十毫秒



(c) 基于时钟中断抢占的优先权抢占调度

2. 立即抢占的优先权调度算法 (更更更快)

- 为所有实时任务设定优先权；
- 高优先权进程到达队列，只要当前进程不在临界区，则立刻抢占CPU；
- 调度时选择队首进程；
- 实时响应时间：数百微秒-数毫秒

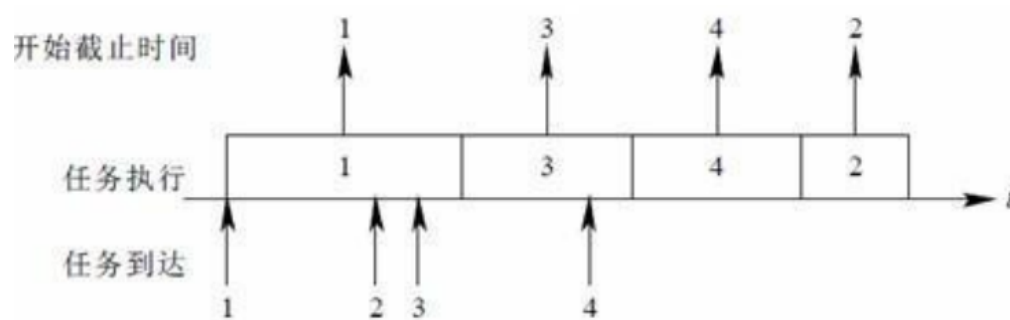


(d) 立即抢占的优先权调度

3.3.3 两个典型的实时调度算法

(1) 最早截止时间优先调度算法(EDF)

- 根据任务的开始截止时间来确定任务的优先级。
- 截止时间愈早，其优先级愈高。
- 该算法要求在系统中保持一个实时任务就绪队列，该队列按各任务截止时间的早晚排序，具有最早截止时间的任务排在队列的最前面。
- 调度程序在选择任务时，总是选择就绪队列中的第一个任务，为之分配处理机，使之投入运行。



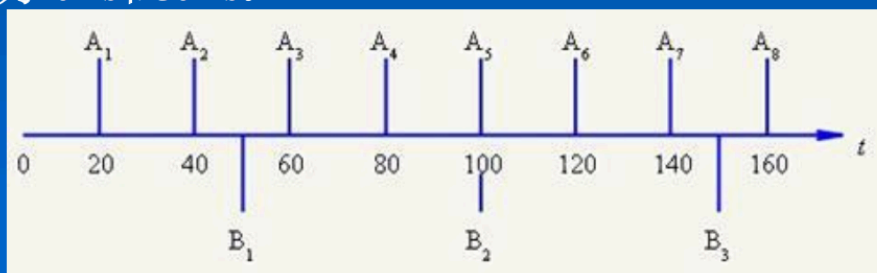
(2) 最低松弛度优先调度算法(LLF)

- 根据任务的紧急程度（松弛度）来确定任务的优先级。
- $\text{松弛度} = \text{必须完成时间} - \text{本身运行时间} - \text{当前时间}$
- 调度程序在选择任务时，总是选择就绪队列中紧急程度（松弛度）最大任务，为之分配处理机，使之投入运行。

算法示例：

■ 算法示例：

周期性任务A和B，执行时间分别为10ms和25ms，周期分别为20ms和50ms。



● 调度过程:

(1) 0ms时刻:

$$A_1 = 20 - 10 - 0 = 10$$

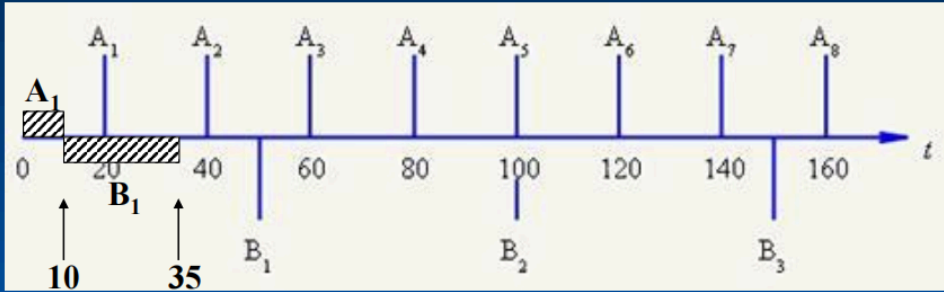
$$B_1 = 50 - 25 - 0 = 25$$

● 调度过程:

(2) 10ms时刻:

$$A_2 = 40 - 10 - 10 = 20$$

$$B_1 = 50 - 25 - 10 = 15$$



● 调度过程:

(3) 30ms时刻:

$$A_2 = 40 - 10 - 30 = 0$$

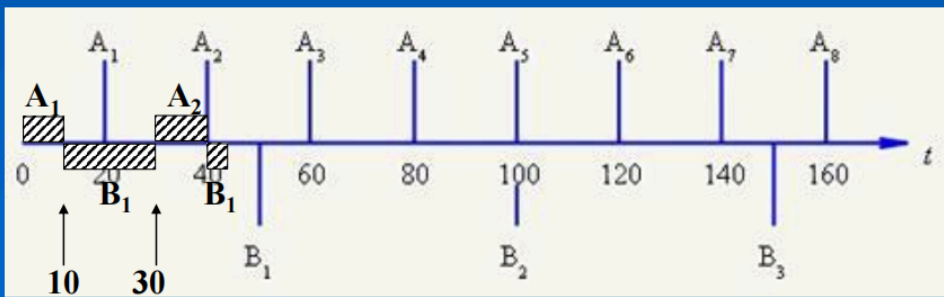
$$B_1 = 50 - 5 - 30 = 15$$

● 调度过程:

(4) 40ms时刻:

$$A_3 = 60 - 10 - 40 = 10$$

$$B_1 = 50 - 5 - 40 = 5$$



3.3.4 优先级倒置问题

(1) 优先级倒置

1. **正常**: 高优先级进程先执行, 低优先级进程后执行
2. **特殊情况**: 低优先级进程先执行, 高优先级进程后执行 (优先级倒置)
 - **后果**: 高优先级进程执行被延迟

(2) 优先级倒置实例

3.3 实时调度

3.3.4 优先级倒置问题

(2) 优先级倒置实例

T1(H):

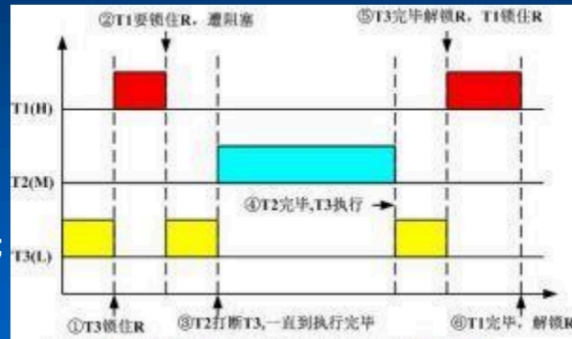
```
wait(mutex);  
critical section;  
signal(mutex);
```

T2(M):

```
remainder section;
```

T3(L):

```
wait(mutex);  
critical section;  
signal(mutex);
```



结果：
低优先级T2比高优先级T1先执行，
使得T1执行延迟。

(3) 优先级倒置的解决方法

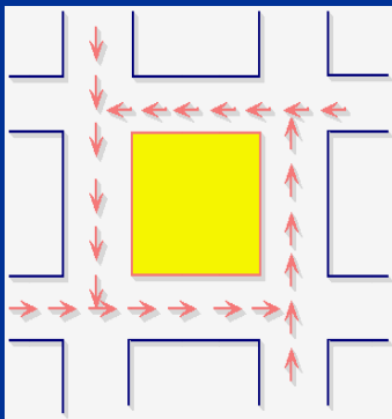
优先权继承：

- 当低优先级进程阻塞高优先级进程
- 将低优先级进程的优先级提高到被它被阻塞进程的最高优先级进程的优先级
- (低优先级进程继承→高优先级进程的优先级)

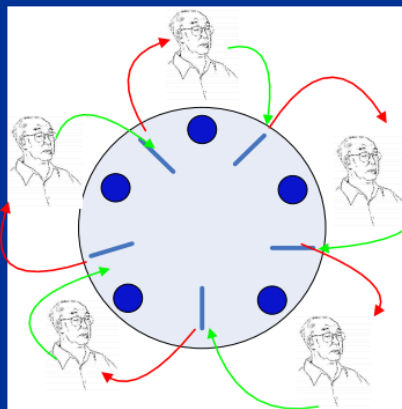
3.5 死锁的基本概念

3.5.1 死锁及死锁状态

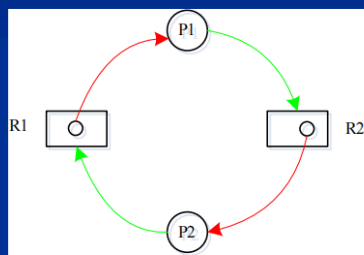
例1：交通死锁



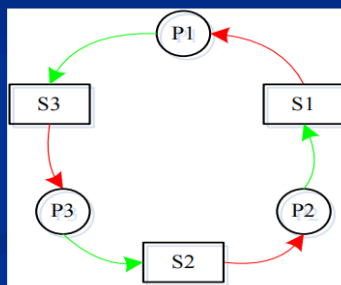
例2：哲学家进餐



例3：两进程共享资源



例4：三进程接发消息



P1: Receive(S3), Send(S1)

P2: Receive(S1), Send(S2)

P3: Receive(S2), Send(S3)

死锁 (Deadlock) 的定义：

- 指多个进程在运行过程中因争夺资源而造成的一种僵局。
- 当进程处于这种僵局时，若无外力作用，他们都将无法再向前推进。

死锁和阻塞的区别：

- 死锁的进程处于阻塞状态，但仅依靠自己，无法继续运行

死锁举例：

(1) 生产者-消费者问题 (错误代码)

■ 从并发进程的代码看死锁1

生产者-消费者问题(错误代码)：

```

VAR mutex, empty, full: semaphore := 1, n, 0 ;
In, out: integer := 0, 0 ;
Buffer: array [0..n-1] of item ;
  
```

Parbegin

Producer:

begin

Repeat

Produce an item in nextp ;

.....

wait(empty) ;

wait(mutex) ;

Buffer(in) := nextp ;

in := (in + 1) mod n ;

signal(mutex) ;

signal(full) ;

Until false

End

Consumer:

begin

Repeat

wait(mutex) ;

wait(full) ;

Nextc = Buffer(out);

out := (out + 1) mod n ;

signal(mutex) ;

signal(empty) ;

Consume the item nextc ;

Until false

End

Parend

出现死锁时机：

empty=n and full = 0 ;

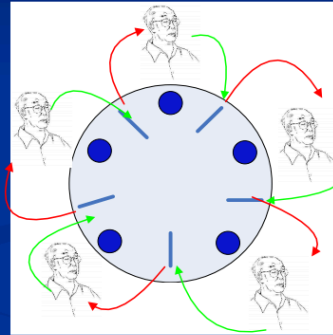
- 当empty = n 且full = 0时，陷入死锁，wait(full)始终阻塞

(2) 哲学家进餐问题 (错误代码)

■ 从并发进程的代码看死锁2

哲学家进餐问题(错误代码):

```
VAR chopstick: array [0..4] of semaphore := 1;  
Parbegin  
  Philospheri:  
    begin  
      Repeat  
        Think ;  
        wait(chopstick[i] ) ;  
        wait(chopstick[(i+1)mod 5] ) ;  
        ...  
        Eat ;  
        signal(chopstick[i] ) ;  
        signal(chopstick[(i+1)mod 5] ) ;  
      Until false  
    End  
  Parend
```



出现死锁时机:
执行第一个Wait后;

- 执行第一个wait后进入死锁状态 (? 什么原因)

3.5.2 产生死锁的原因

1. 竞争非剥夺性资源

- 可剥夺性资源 (未用完可以被剥夺): 内存等。
- 非剥夺性资源 (必须在用完后才能被剥夺): 打印机等

2. 进程推进顺序非法

- 错误的进程推进顺序 导致 死锁。
- 正确的进程推进顺序 不导致 死锁。

3.5.3 产生死锁的必要条件——死锁的特征

1. 互斥条件

- 互斥使用资源: 资源自身的特点

2. 请求和保持条件

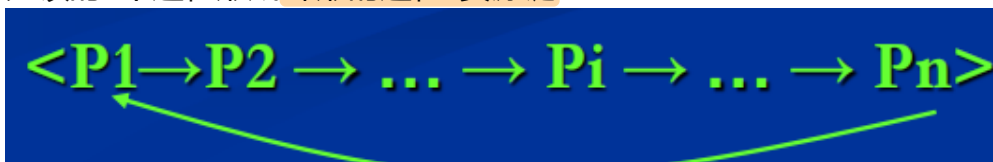
- 进程占有一个资源的同时, 请求另外的资源

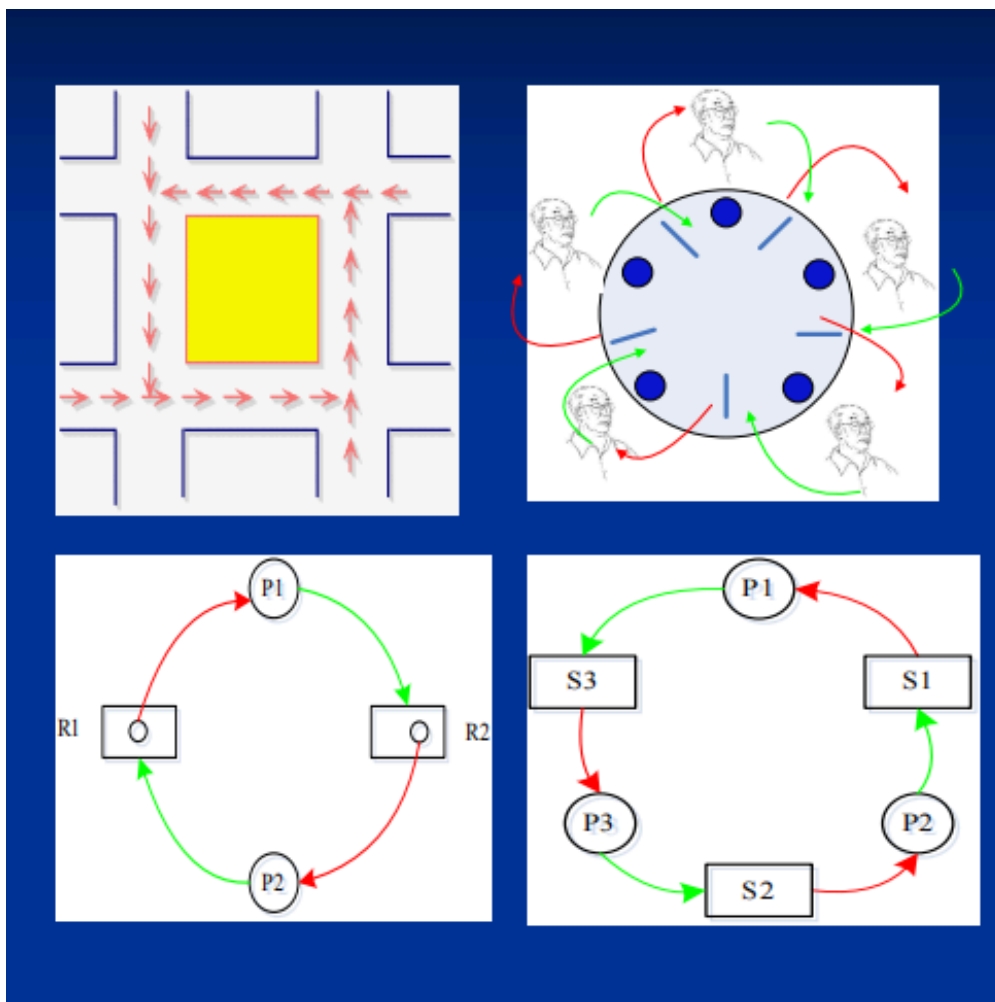
3. 不剥夺条件

- 进程的资源使用完之前, 不能被剥夺

4. 环路等待条件

- 死锁的n个进程形成环形的进程-资源链





3.5.4 处理死锁的基本方法

(1) 预防死锁

- **作法:** 破坏死锁产生的四个必要条件
- **优点:** 简单;
- **缺点:** 资源利用率低, 效率低

(2) 避免死锁

- **作法:** 进程申请资源时, 由系统审查申请的合理性, 只有不导致死锁的申请, 才被认可。
- **优点:** 效率略高。
- **缺点:** 实现困难: 进程对资源的需求不好事先确定;

(3) 检测和解除死锁

- **作法:** 进程申请资源时不作任何限制, 仅在系统中定时或资源不足时, 检查是否有死锁; 如果有, 解决之。
- **优点:** 不影响系统执行性能, 效率高。

3.6 预防死锁

3.6.1 预防死锁的基本概念

1. 基本思路

- 死锁→四个基本条件
- 逆否命题：四个基本条件不同时成立→无死锁

2. 作法：

- 破坏死锁产生的四个基本条件

3.6.2 破坏“互斥”条件（不可能）

- 方法：不可能！资源特性界定，否则程序运行结果不可再现

3.6.3 破坏“请求和保持”条件

- 方法：进程创建时，一次申请所有资源。成功则运行，否则阻塞。（这样一来进程就不需要在运行时请求其他资源了）
- 优点：简单、易于实现、安全
- 缺点：
 - 资源严重浪费
 - 进程执行进度大大延迟

3.6.4 破坏“不剥夺”条件

- 方法：进程申请资源时，成功则运行，否则释放所有资源后阻塞。
- 优点：无
- 缺点：
 - 资源严重浪费
 - 代价太大
 - 进程执行进度被严重延迟

3.6.5 破坏“环路等待”条件

- 方法：
 - 为所有资源编号，进程申请资源时必须按序申请资源。
 - 例如：
 - R1: 打印机, R2: 磁带机; R3: 磁盘; R4: 输入机
 - 进程P1使用资源顺序: 输入机R4, 磁带机R3, 打印机R1, 磁盘R2
 - 但是P1 资源申请 顺序必须为: R1,R2,R3,R4 （编号顺序确定好了）
- 优点：资源利用率、系统吞吐量显著提高
- 缺点：资源还是会浪费；资源编号困难；新资源加入困难

3.7 避免死锁

3.7.0 避免死锁的基本思想

1. 允许系统具备四个必要条件
2. 进程在执行过程中动态地申请和释放资源
3. 每次资源分配时，由系统仔细检查此次资源分配的安全性，只有在安全时才分配，否则不分配，并令进程等待。（银行家算法）

3.7.1 安全状态与不安全状态

(1) 定义

假设/前提：假定某系统有n个进程并发执行，对于某时刻T0

定义：

- 系统能够按照某种进程顺序 $\langle P_1, P_2, \dots, P_n \rangle$ 执行
- 当轮到每个进程 P_i 执行时，都能满足该进程对资源的最大需求
- 则该系统处于安全状态，否则为不安全状态

安全序列： $\langle P_1, P_2, \dots, P_n \rangle$

(2) 示例

安全状态：可用资源顺序分配给P2、P1、P3，每次分配给某个进程执行完成后回收该进程所有资源，可以发现不会出现资源短缺问题

2. 安全状态与不安全状态示例

某系统3个进程P1,P2,P3，共享12台磁带机资源，某时刻T0的资源分配状况如下表所示：

进程	最大需求量	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

(1) T0时刻安全性

安全序列： $\langle P_2, P_1, P_3 \rangle \rightarrow$ 安全状态

不安全状态：动态地把一个可用资源分配给3，剩余2个，发现怎么分配都是会短缺的，不存在安全序列，故处于不安全状态

(2) 安全状态转化为不安全状态

进程	最大需求量	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

P3请求1台磁带机



进程	最大需求量	已分配	可用
P1	10	5	2
P2	4	2	
P3	9	3	

安全序列: $\langle P2, ?, ? \rangle \rightarrow$ 不安全状态

(3) 两种状态的关系

1. 系统处于安全状态，不会死锁；
2. 系统处于不安全状态，可能死锁；
3. 系统在安全状态与不安全状态之间不断转换

3.7.2 银行家算法 (算法重点)

(1) 问题

- n 个进程 $P_1, P_2, \dots, P_n$
- 共享 m 类资源 $R_1, R_2, \dots, R_m$
- 进程并发执行时，如何避免死锁

(2) 数据结构设置

名称	长度/形状	含义
Available	向量长度 m	可用资源向量，Available[j]为资源 R_j 的可用数量
Max	n 行 m 列	最大需求矩阵，Max[i][j]表示进程 P_i 对资源 R_j 的最大需求量
Allocation	n 行 m 列	分配矩阵，Allocation[i][j]表示进程 P_i 已经分到的资源 R_j 数量
Need	n 行 m 列	需求矩阵，Need[i][j]表示进程 P_i 对资源 R_j 的需求数量
Request	向量长度 m	资源请求向量，Request _{i} [j]表示进程 P_i 对资源 R_j 的请求数量

(3) 银行家算法 (进程 P_i 请求Request _{i})

步骤如下：

1. **判断请求数量是否小于等于需求的数量** if $Request_i \text{ not } \leq Need_i$ Then 出错
2. **判断请求数量是否可分配**: if $Request_i \text{ not } \leq Available$ Then P_i 等待
3. **试分配**: 修改数据结构Allocation,Need,Available
 - $Available := Available - Request_i$
 - $Allocation_i := Allocation_i + Request_i$
 - $Need_i := Need_i - Request_i$
4. **安全状态判断**: 执行 **安全性算法 (见4)**, 检查此次资源分配后系统是否处于安全状态
5. **结果**: if 安全 Then 正式分配 Else 取消试分配并使 P_i 等待

(4) 安全性算法

步骤如下:

1.
 - 设置**工作向量Work**, 长度为m, $Work := Available$
 - 设置**状态向量Finish**, 长度为n, $Finish := false$
2. 从进程集合中查找满足下列条件的进程 P_k (**看有没有进程可分配**)
 - $Finish[k] = false$
 - $Need_k \leq Work$
 - 如果均不符合, 跳转步骤4 (**进程需求均大于可分配, 不安全**)
3. 执行如下操作
 - $Work := Work + Allocation_k$
 - $Finish[k] := true$;
 - 回到步骤2循环
4. 如果 **Finish = true**, 那么安全, 否则不安全

(5) 例题

■ 进程{P0,P1,P2,P3,P4}共享资源{A,B,C}。某T0时刻资源状况如下：

	Max	Allocation	Need	Available
	A B C	A B C	A B C	A B C
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

■ 问题：

- (1) T0时刻安全性；
- (2) P1请求Request₁(1,0,2)；
- (3) P4请求Request₄(3,3,0)；
- (4) P0请求request₀(0,2,0)；

具体解答见书p113，需要列表求解

(6) 总结

- 银行家算法提供比死锁预防更好的进程并发度；
- 需要提前预知进程对各类资源的最大需求数量，在很多情况下几乎不可能。

(7) 思考题

1. 3个进程共享资源A，每个进程最大需求为2个，则A至少有多少个资源时，才永远不会产生死锁
2. n个进程共享资源A，每个进程最大需求为x个，则A至少有多少个资源时，才永远不会产生死锁？

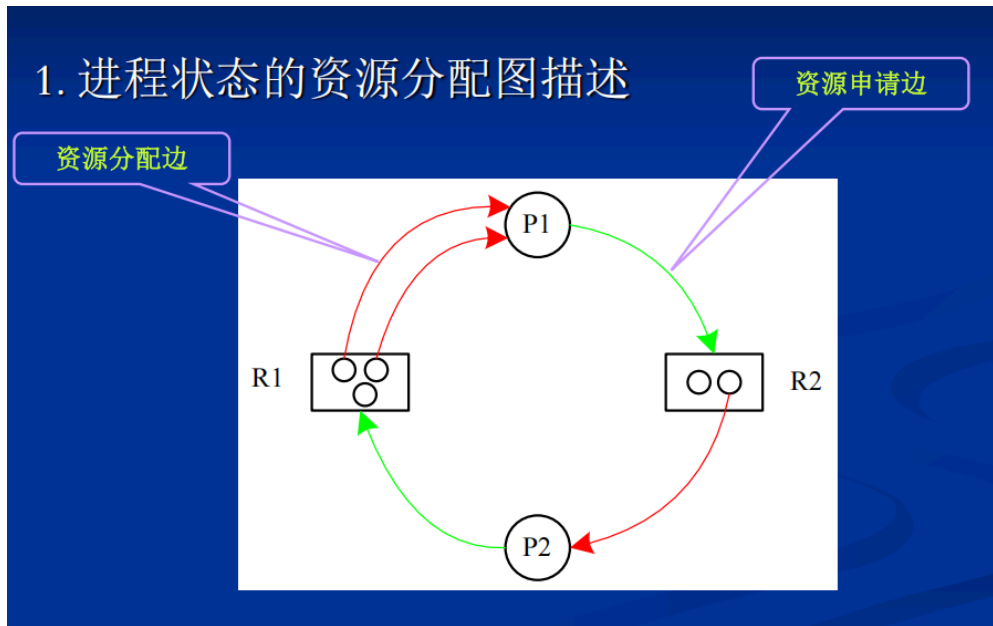
3.8 死锁的检测和解除

3.8.0 基本思想

1. 进程在执行过程中动态地申请和释放资源
2. 每次资源分配时，不作任何检查；
3. 允许系统产生死锁；
4. 定时或按需检查系统是否有处于死锁状态的进程存在；
5. 发现死锁，则立即解除所有死锁进程

3.8.1 死锁的检测

(1) 资源分配图



图中：

- $P_i \rightarrow R_j$ ：资源申请边—— P_i 向 R_j 申请资源
- $R_j \rightarrow P_i$ ：资源分配边—— R_j 向 P_i 分配资源

(2) 死锁检测

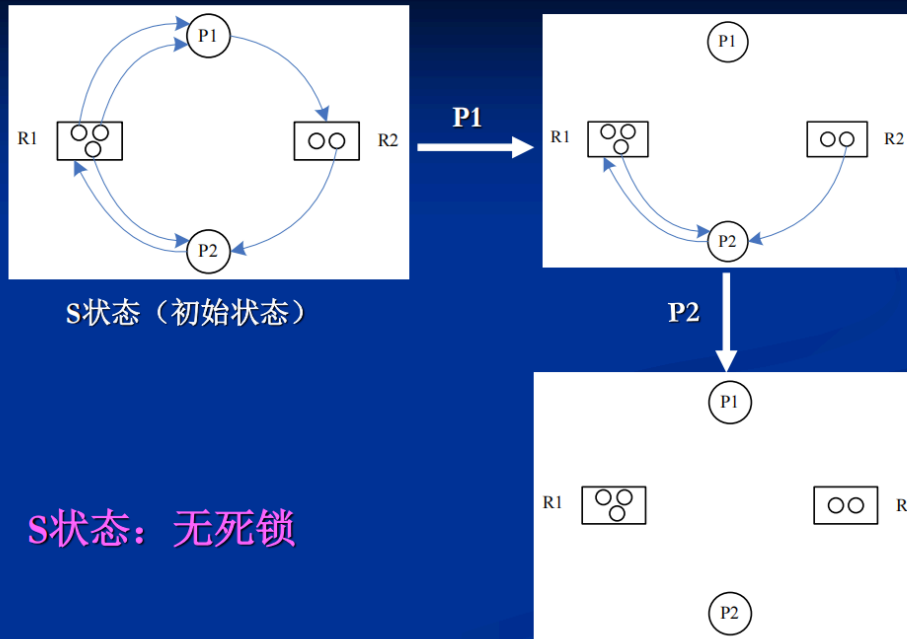
步骤如下：

1. 对T0时刻资源分配图，寻找一个非阻塞非孤立结点，找不到则跳转步骤4
 - 不阻塞的含义： R_j 有足够的资源分配给进程 P_i ，即进程 P_i 的入边是可以实现的
2. 删除与该节点相连的所有边
3. GOTO(1)循环
4. if 最终资源分配图中均为孤立节点 Then 无死锁 Else 有死锁

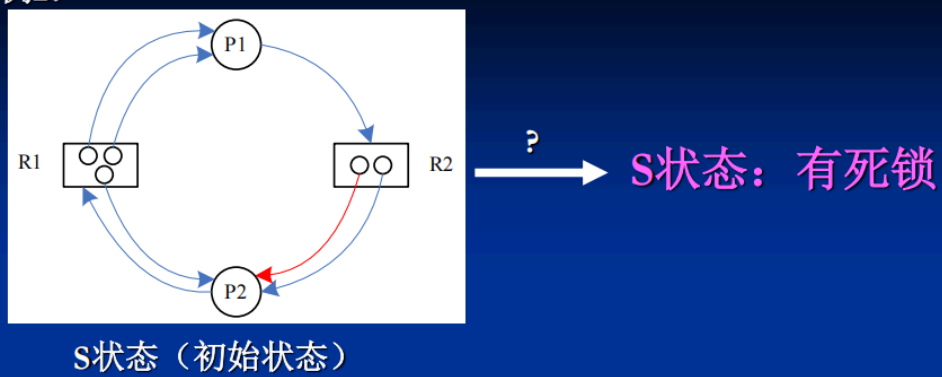
主要方法： R_j 向外的箭头就是已经分配的资源，需要逐步消除 P_i 归还资源

例题：例4应该是真死锁，所有进程的资源申请都无法满足

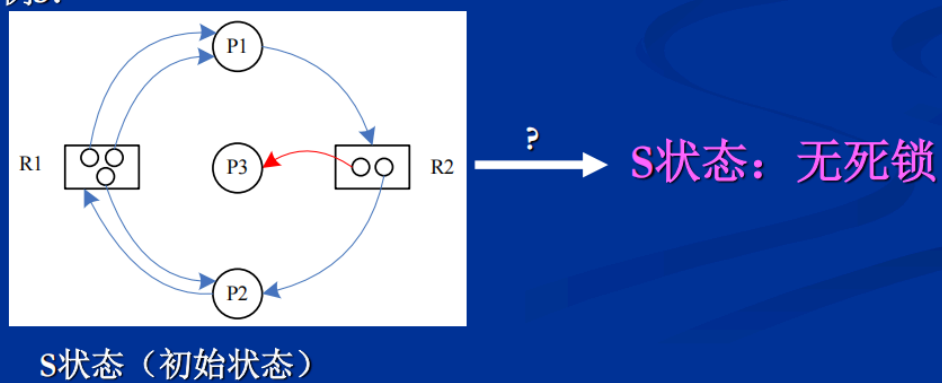
例1:



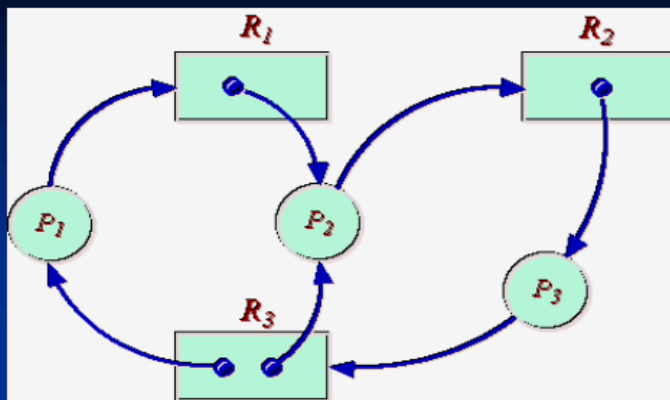
例2:



例3:



例4:

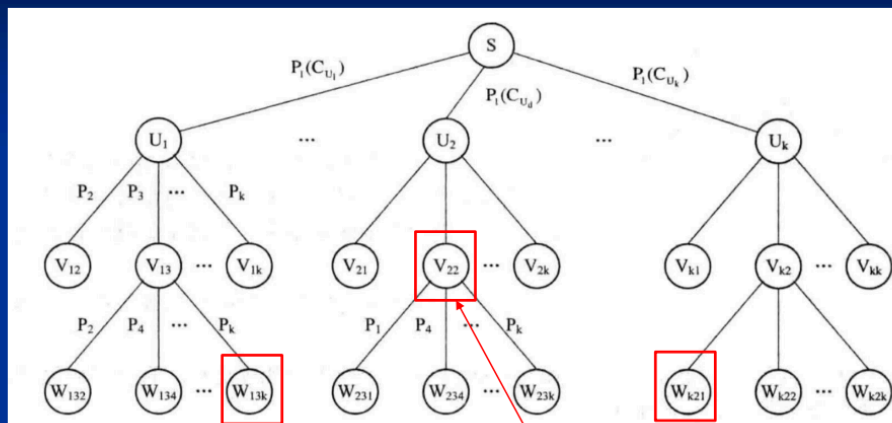


3.8.2 死锁的解除

基本解除策略:

- (1) 剥夺资源
- (2) 撤消进程
 - 采用深度进程策略撤消
 - 采用宽度进程策略撤消

■ 撤消进程: 广度($O(2^n)$)vs.深度($O(2^n)$)



可解除死锁的状态

时间复杂度: 深度/广度($O(2^n)$) vs. 启发式深度($O(n^2)$)